

Practical 1

Government College of Engineering, Jalgaon (An Autonomous Institute of Govt. of Maharashtra) Department of Computer Engineering	
Name:	
PRN:	Batch:
Class:	Semester:
Course:	Subject Teacher:
Date of Performance:	Date of Completion:

Aim: Program for Distributed Application using RMI

Theory :

RMI (Remote Method Invocation)

The **RMI** (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. The RMI allows an object to invoke methods on an object running in another JVM.

The RMI provides remote communication between the applications using two objects *stub* and *skeleton*.

Understanding stub and skeleton-

RMI uses stub and skeleton object for communication with the remote object.

A **remote object** is an object whose method can be invoked from another JVM. Let's understand the stub and skeleton objects:

Stub

The stub is an object, acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes method on the stub object, it does the following tasks:

1. It initiates a connection with remote Virtual Machine (JVM),
2. It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
3. It waits for the result
4. It reads (unmarshals) the return value or exception, and

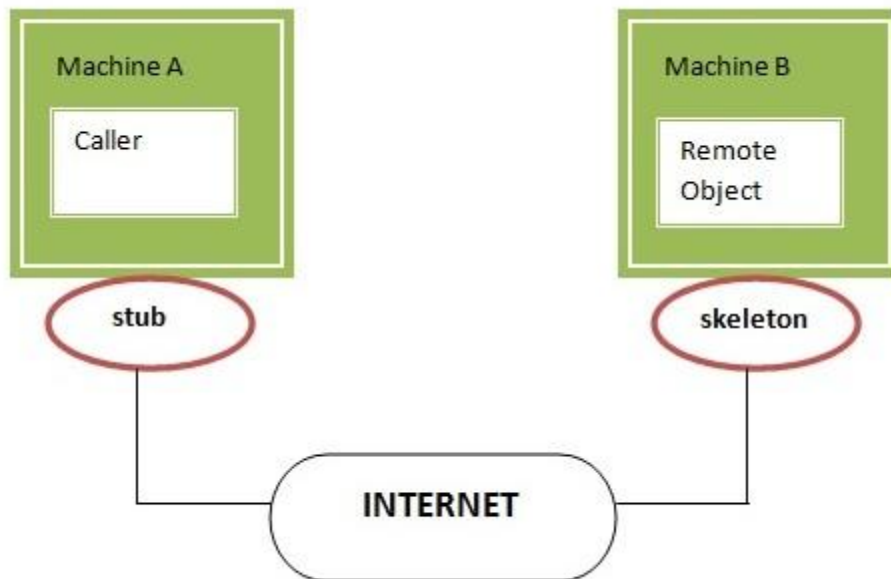
5. It finally, returns the value to the caller.

Skeleton-

The skeleton is an object, acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:

1. It reads the parameter for the remote method
2. It invokes the method on the actual remote object, and
3. It writes and transmits (marshals) the result to the caller.

In the Java 2 SDK, an stub protocol was introduced that eliminates the need for skeletons.



Understanding requirements for the distributed applications

If any application performs these tasks, it can be distributed application.

1. The application need to locate the remote method
2. It need to provide the communication with the remote objects, and
3. The application need to load the class definitions for the objects.

The RMI application have all these features, so it is called the distributed application.

Java RMI Example

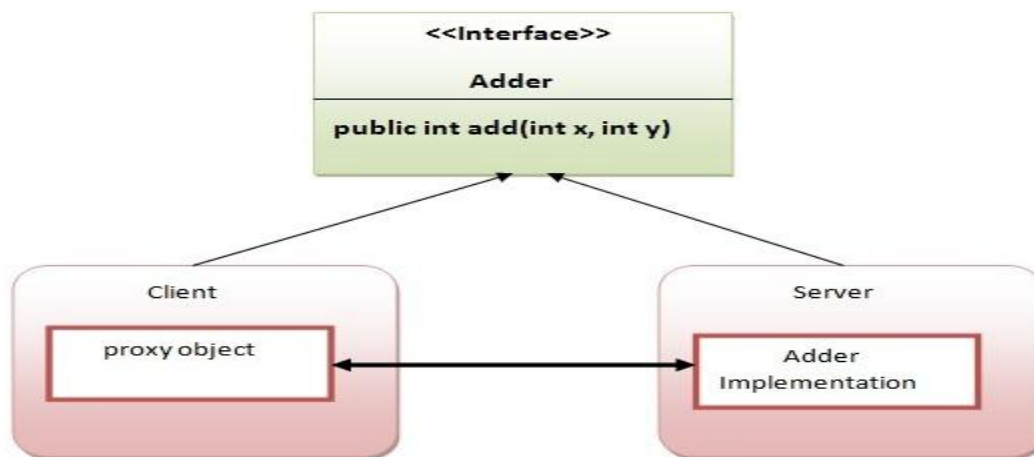
The is given the 6 steps to write the RMI program.

1. Create the remote interface
2. Provide the implementation of the remote interface
3. Compile the implementation class and create the stub and skeleton objects using the rmic tool

4. Start the registry service by rmiregistry tool
5. Create and start the remote application
6. Create and start the client application

RMI Example

In this example, we have followed all the 6 steps to create and run the rmi application. The client application need only two files, remote interface and client application. In the rmi application, both client and server interacts with the remote interface. The client application invokes methods on the proxy object, RMI sends the request to the remote JVM. The return value is sent back to the proxy object and then to the client application.



1) create the remote interface

For creating the remote interface, extend the Remote interface and declare the RemoteException with all the methods of the remote interface. Here, we are creating a remote interface that extends the Remote interface. There is only one method named add() and it declares RemoteException.

2) Provide the implementation of the remote interface

Now provide the implementation of the remote interface. For providing the implementation of the Remote interface, we need to

Either extend the UnicastRemoteObject class,

or use the exportObject() method of the UnicastRemoteObject class

In case, you extend the UnicastRemoteObject class, you must define a constructor that declares RemoteException.

3) create the stub and skeleton objects using the rmic tool.

Next step is to create stub and skeleton objects using the rmi compiler. The rmic tool invokes the RMI compiler and creates stub and skeleton objects.

```
rmic AdderRemote
```

4) Start the registry service by the rmiregistry tool

Now start the registry service by using the rmiregistry tool. If you don't specify the port number, it uses a default port number. In this example, we are using the port number 5000.

```
rmiregistry 5000
```

5) Create and run the server application

Now rmi services need to be hosted in a server process. The Naming class provides methods to get and store the remote object. The Naming class provides 5 methods.

6) Create and run the client application

At the client we are getting the stub object by the lookup() method of the Naming class and invoking the method on this object. In this example, we are running the server and client applications, in the same machine so we are using localhost. If you want to access the remote object from another machine, change the localhost to the host name (or IP address) where the remote object is located.

Code :

Server.java

```
import java.sql.*;
import java.sql.Connection;
import java.rmi.*;
import java.rmi.Naming.*;
import java.rmi.server.*;
import java.rmi.registry.*;
import java.util.Vector;
interface DBInterface extends Remote {
    public String input(String name1, String name2) throws RemoteException;
}
public class Server_Practical_1 extends UnicastRemoteObject implements DBInterface {
    int flag = 0,n,i,j;
    String name3;
    ResultSet r;

    public Server_Practical_1() throws RemoteException {
        try {
            System.out.println("Initializing Server\nServer Ready");
        } catch (Exception e) {
            System.out.println("ERROR: " + e.getMessage());
        }
    }

    public static void main(String [] args) {
        try {
            Server_Practical_1 rs = new Server_Practical_1();
            java.rmi.registry.LocateRegistry.createRegistry(1030).rebind("DBServ", rs);
        } catch (Exception e) {
            System.out.println("ERROR: " + e.getMessage());
        }
    }

    public String input(String name1, String name2) {
        try {
            name3 = name1.concat(name2);
        } catch (Exception e) {
            System.out.println("ERROR: " + e.getMessage());
        }
        return name3;
    }
}
```

Client.java

```
import java.sql.*;
import java.rmi.*;
import java.io.*;
import java.util.*;
import java.util.Vector.*;
import java.lang.*;
import java.rmi.registry.*;
public class Client_Practical_1 {
    static String name1,name2,name3;
    public static void main(String args []) {
        Client_Practical_1 c = new Client_Practical_1();
        BufferedReader b = new BufferedReader(new InputStreamReader(System.in));
        int ch;
        try {
            Registry r1 = LocateRegistry.getRegistry ("localhost", 1030);
            DBInterface DI =(DBInterface) r1.lookup("DBServ");
            do {
                System.out.print( "\n\n1.send input strings\n2.Displayconcatenated string \nEnter ur
choice :- ");
                ch = Integer.parseInt(b.readLine());
                switch(ch) {
                    case 1: System.out.print("\nEnter first string: ");
                        name1 = b.readLine();
                        System.out.print("\nEnter second string: ");
                        name2 = b.readLine();
                        name3 = DI.input(name1, name2);
                        break;

                    case 2: // display System.out.println("\n Concatenated String is : ");
                        int i = 0;
                        System.out.println("Concatenated String :- " + name3);
                        break;
                } } while(ch > 0);
            }
            catch (Exception e) {
                System.out.println("ERROR: " + e.getMessage());
            }
        }
    }
}
```

Output :

Server.java

```
D:\Me\College\DSL\1_Distributed_RMI>javac Server.java

D:\Me\College\DSL\1_Distributed_RMI>java Server
Initializing Server
Server Ready
█
```

Client.java

```
D:\Me\College\DSL\1_Distributed_RMI>java Client
1. Send input strings
2. Display concatenated string
Enter your choice
1

Enter first string:
Distributed

Enter second string:
Systems

1. Send input strings
2. Display concatenated string
Enter your choice
2

Concatenated String is:
DistributedSystems
1. Send input strings
2. Display concatenated string
Enter your choice
█
```

Conclusion : In this practical we learnt about RMI and its implementation

Sign of teacher
Mrs. Archana Chitte.